WIRELESS SECURITY · MACHINE LEARNING

Catching the Deauth Flood

A lightweight Wi-Fi intrusion detection & prevention system that learns the difference between normal 802.11 traffic and a management-frame flood — and blocks the attacker automatically.

WIDS / WIPS prototype
scikit-learn · Scapy · Python
Random Forest classifier
Accuracy 99.5% on unseen attacks
Lab/testbed use only

01 How the system works

Attacks like a **deauthentication flood** are not one strange frame — they are an abnormal *burst* of management frames from one device. So the system groups frames into 1-second windows per transmitter, measures a few simple things, and lets a classifier decide.

CAPTURE
Read 802.11 frames
A .pcap file, or a live monitor-mode interface. Same code path.

→

FEATURES
Per device / second
14 simple numbers: frame rates, type mix, signal strength, spread.

→

CLASSIFY
Random Forest
Labels each window normal or malicious.

→

RESPOND
Alert → Block
WIDS logs an alert; WIPS blocklists the offending MAC.

`rssi_std` `frac_broadcast` `deauth_per_sec` `frames_per_sec` `disassoc_per_sec` `frac_mgmt / ctrl / data` `rssi_mean / min` `n_unique_dst` `mean_size`

02 The data

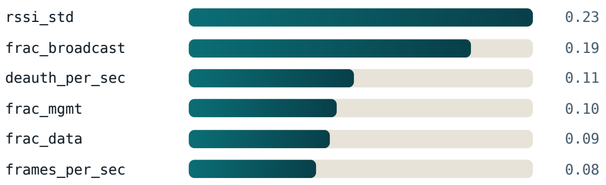
20 synthetic capture scenarios built with Scapy, each ~20 s of a small Wi-Fi network: an access point, 2–5 clients browsing and streaming, handshakes and power-save frames.

- **Normal traffic** includes a few *legitimate* deauth/disassoc frames (a client leaving), so the model can't cheat with "any deauth = attack".
- **Attacks**: broadcast flood, targeted flood, low-rate stealth flood, disassociation flood, rogue-MAC flood — each labelled per frame.
- Every frame carries a RadioTap **signal strength** (dBm) and a ground-truth label.

> **Ethics:** all frames are generated in memory and written to files. Nothing is transmitted over the air. Live capture / attack generation is for an authorised, isolated testbed only.

03 Why these features win

Feature importances from the trained Random Forest. A flood gives itself away three ways: it is **broadcast-heavy**, its deauth rate **spikes**, and the spoofing device's **signal wobbles** more than a settled client's.



Top 6 of 14 features. The forest spreads credit across correlated signals, so no single feature dominates. Exact values are printed by `train.py` and saved to `results/feature_importance.png`.

04 Results

Tested by holding out **whole capture scenarios** (a group split), so the model is scored on attacks it never saw in training — no leakage.

99.5%	1.00	0.88	0
Accuracy	Precision (attack)	Recall (attack)	False alarms

05 Confusion matrix

Held-out test windows (rows = actual, columns = predicted).

	PRED NORMAL	PRED ATTACK
ACTUAL NORMAL	421 TRUE NORMAL	0 FALSE ALARM
ACTUAL ATTACK	2 MISSED	14 CAUGHT

The 2 misses are the first/last window of a short attack, when only a handful of flood frames fall inside the window. A single Decision Tree scored 0.75 recall; the Random Forest raises it to 0.88 with still zero false alarms.

06 Live detection & automatic response

Real-time replay of an attack capture. Normal traffic scrolls by; the moment the flood starts, the window's `deauth/s` jumps and the classifier fires. In WIPS mode the offending MAC is blocklisted automatically.

```
  time  device              frm/s  deauth/s  signal  status
  10.0  1c:2e:2b:b8:56:9d      54         0   -47dBm  ok
  11.0  1c:2e:2b:b8:56:9d     164       103   -57dBm  *** ATTACK ***
        -> WIPS blocked 1c:2e:2b:b8:56:9d
  12.0  1c:2e:2b:b8:56:9d     194       144   -59dBm  *** ATTACK ***
  13.0  10:0e:ba:ee:a3:c2      62         0   -61dBm  ok  (real client, unaffected)
  14.0  1c:2e:2b:b8:56:9d     195       143   -59dBm  *** ATTACK ***
  ----------------------------------------------------------------
Attack mitigated. Blocked: ['1c:2e:2b:b8:56:9d']  ·  WIPS would run:
iptables -A INPUT -m mac --mac-source 1c:2e:2b:b8:56:9d -j DROP
```

07 What I'd improve

- Train and validate on a **real public dataset** (AWID3) and on live captures from an authorised lab, not only synthetic traffic.
- Add more attack classes: evil-twin / rogue AP, beacon flood, authentication flood, KRACK-style.
- Make it **multi-class** (name the attack) instead of binary, and add a probability threshold to trade recall against false alarms.
- Shrink the window / use overlapping windows to catch the first second of an attack (the current misses).
- Stream features continuously and push alerts to a dashboard; rate-limit the auto-block and add a manual un-block.

08 Use of AI assistants

An AI coding assistant (Claude Code) helped **scaffold the Python modules**, explain Scapy's 802.11 frame fields and the scikit-learn API, and draft this poster.

I made the **design decisions** myself: per-device time-window features rather than per-frame; including legitimate deauths in the normal class; and switching from a random split to a **group split** after noticing the random split leaked windows and inflated the score. I verified every number by running the code and reading the confusion matrix, and I can walk through how each part works in the demo.

Reproduce: `pip install -r requirements.txt` → `bash run_all.sh` — generates data, trains, evaluates, and runs the WIDS + WIPS demos.

Detects: deauth · disassoc · broadcast / targeted / stealth / rogue-MAC floods